

Stepping up MetaPost

convenience and performance

context 2022 meeting

MetaPost version 2 versus 3

- There is less need to adapt MetaPost than $\text{T}_{\text{E}}\text{X}$ to current developments, most needed additional can be dealt with in the backend via the generic pre- and postscript properties. These properties have been added to some more data structures.
- Some features of MetaPost are not needed or limited so these are removed: fonts, backends. Only three number models are supported by default: legacy scaled, double, and decimal. We default to double, which also performs best.
- Most significant are the extensions in the Lua interface where we got powerful and fast scanners. This permits us to extend the language.
- However, we also added some primitive features, simply because it removed some persistent bottlenecks. The internals didn't really change much conceptually. There are numerous optimizations, so it runs faster anyway.
- For Mikael and me this was a nice distraction from dealing with the messy math fonts.

Intersection

Getting all intersections of two paths is a bit difficult. It also involves repetitive restart and cutting before and after and hoping for the best. This is why we now have

```
1 <path> intersectiontimeslist <path>
```

This operator returns a path made up of points that reflect the times on the paths.

```
1 intersections-001.tex
```

Loops

- A path is a linked list which means that getting to a point (or between points) is a linear lookup.
- We now have a double linked list and support for progressive loops over paths.
- It is not pretty but works ok for what we need and often the dirty details are hidden in macros.
- New primitives are: `deltapoint`, `deltaprecontrol`, `deltapostcontrol`, `deltadirection`, `pathpoint`, `pathprecontrol`, `pathpostcontrol`, `pathdirection`.

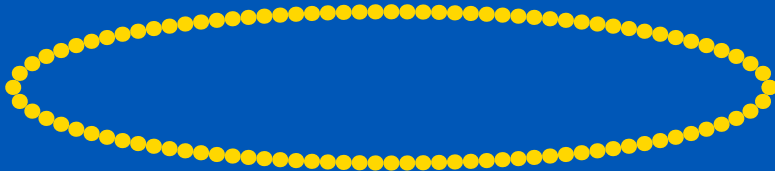
1 intersections-001.tex

Arcs

The agitate article by Fabrice Larribe was a good reason to see if we could bring down many hours of rendering to minutes. For that we came up with `arcpoint`, `arcpointlist` and `subarclength`.

```
1 path p ; p := arcpointlist 100 of (fullcircle xyscaled (10cm,2cm)) ;  
2 draw image ( for i within p :  
3   draw pathpoint withpen pencircle scaled 2mm ;  
4 endfor ; ) withcolor "extracolor" ;
```

This gives equally spread points as path:



```
1 agitate-002.tex  
2 ontarget-metapost.tex
```

Drawing features

Instead of grouping with interims for line properties we can also use:

```
<path> withlinecap <numeric>  
2 <path> withlinejoin <numeric>  
3 <path> withlinemiterlimit <numeric>
```

This gives less overhead and is more convenient in generated MetaPost code.

Paths

The macro center has two primitive companions:

`centerof <path>`

`centerofmass <path>` (Mikael can explain that one)

These avoid multiple calculations of corners of the bounding box we have:

`corners <path|picture>`

`boundingpath <pen> of <path>`

`xrange <path|picture>`

`yrange <path|picture>`

For generated code or less complex macros we have:

`nocycle`

and we have `&&` for connecting no-continuous paths (as in fonts) but that is still experimental.

Pens

Pens can be tricky: circular pens are native PostScript (and pdf) but when transformed are handled by double paths. Transparency can uncover all kind of side effects that are normally hidden. The envelopes made by pens are approximations that assume that these inaccuracies end up inside the filled areas. Pens get convexed.

```
1 fill convexed (fullcircle xysized (10cm,1cm))  
2   withcolor "extracolor" withpen pencircle scaled 1mm ;
```



The problem is that once we figure this out and apply it low level it's easy to forget all the details. This is not everyday code and it's hard to make examples.

A good way out of inaccuracies is to rotate over eps which avoids some zero conditions. The envelope primitive has been improved and for special tracing nep and makenep are introduced to get around some heuristics.

```
1 ontarget-envelopes.tex
```

Lua

There are plenty of new/improved scanners and injectors so that we can build interfaces that are very efficient. Given the performance of MetaPost we can extend in Lua. In ConT_EXt most interfaces between T_EX and MetaPost are upgraded.

Etcetera

We just mention `overload`, `stacking`, `uncontrolled` and `setgroup` and there are some performance related settings. And we're adding some more in the coming years, like additional support for envelopes.

Mikael Sundqvist made some beautiful, more detailed, overviews of features!